

INTERNSHIP WORK REPORT

Veda — AI-Powered PDF to Interactive Presentation

Company: ADS Softek

Name: Dara Eakeswar Rayudu

Role: AI Intern

Period: 29 June 2026 – 21 July 2026 (four working weeks)

Stack: React 19, Vite, Tailwind CSS v4, FastAPI, Qwen 2.5, NLLB-200, Sarvam AI, Kaggle GPU (T4 × 2)

1. Executive Summary

Veda converts an uploaded PDF into an interactive, narrated presentation with karaoke-style (read-along) word highlighting. During this internship I took a working prototype and turned it into a dual-mode production-style system: a laptop orchestrator for the UI, TTS, and routing, plus optional Kaggle GPU offload for layout, LLM analysis, Telugu translation, and decorative image generation.

The work mixed product decisions, frontend UX, PDF layout engineering, model routing, and operational reliability (tunnels, pre-warm, fallbacks). Constraints were explicit from week one: no dedicated GPU on the local machine, limited laptop CPU, and Kaggle GPU hours that are finite. Those constraints shaped every major architecture choice.

This report is based on the project's Cursor chat history on the local workspace, git commit history, and the maintained architecture notes in .cursorrules. Most heavy ML work after 1 July was carried out on the RDP development machine after the local laptop proved insufficient.

2. Project Objective

Build a web application that:

- Accepts a PDF upload.
- Extracts per-page layout (text lines, columns, images).
- Uses an LLM to produce slide JSON (title, subtitle, summary, highlights, narration).
- Speaks the narration with word-level highlighting synced to audio.
- Supports English and Telugu.
- Presents images clearly (upscale or enhance where appropriate).
- Remains usable when cloud GPU or paid TTS APIs are unavailable.

3. Approach

I approached the internship as an incremental pipeline build, not a single rewrite.

3.1 Understand the product before changing models

Week 1 started by reading the existing architecture, running backend and frontend locally, and mapping which APIs were actually used (Sarvam for TTS/STT and translation; OpenAI Whisper only for optional /transcribe; ElevenLabs listed but unused). That avoided spending time on unused vendors.

3.2 Frontend first while GPU was unavailable

On 1 July it was clear that the laptop could not host heavy inference. I therefore invested in UI consistency, theming, loading states, and Telugu rendering so the product felt complete while backend GPU work waited for the RDP/Kaggle environment.

3.3 Dual-mode from the start of GPU work

Rather than requiring Kaggle for every run, I designed laptop + Kaggle with automatic CPU fallback. Demos never fully stopped when the Cloudflare tunnel died or weekly GPU quota ran out.

Workload	When Kaggle is on	Fallback
Page layout	pdfplumber + PyMuPDF on the notebook	Local page_layout_service.py
Slide analysis	Qwen 2.5 7B on GPU	Local Qwen 2.5 3B
Telugu translation	NLLB-200 600M on GPU	Local NLLB-200 1.3B, then Sarvam
Decorative images	SSD-1B img2img	Lanczos on laptop
Informational images	Always Lanczos on laptop	Same
TTS / karaoke	Always laptop (Sarvam → edge-tts)	Same

3.4 Measure on real magazines, not toy PDFs

Layout and image work was validated against a 97-page Discovery magazine PDF (covers, two-column spreads, Q&A callouts, photo pages). Heuristics were tuned against that corpus rather than synthetic pages.

3.5 Fail closed, then fail over

Paid APIs and GPU tunnels fail often. The pattern used throughout: detect the failure, notify the user where it matters (for example Sarvam credits), and continue with a defined fallback (edge-tts, local NLLB, Lanczos). Silent success with the wrong language was treated as a bug, not an acceptable fallback.

4. Weekly Work Log

Calendar weeks are Monday–Sunday. Dates follow git history and local chat timestamps.

Week 1 — 29 June to 5 July 2026

Theme: Onboarding, bilingual correctness, UI system, move to a stronger machine

What I did

- Created a proper .gitignore so models, vendor/, .env, and generated PDFs would not be committed.

- Documented clone/setup so a collaborator could install Python vendor deps and Node independently of my machine.
- Ran the stack locally (FastAPI :8765, Vite :5173) and confirmed health checks.
- Audited API keys: Sarvam is the live TTS/STT/translate path; OpenAI is optional Whisper; ElevenLabs was unused leftover configuration.
- Fixed Telugu: selecting Telugu did not change on-screen text or narration. Root causes included translating only narration_te, caching incomplete English decks, and not re-running translation on language switch. I extended translation to slide fields, fixed cache invalidation, and later moved primary translation to NLLB-200 with Sarvam as last resort. Added Noto Sans Telugu and a .lang-te class for legible Telugu typography.
- Migrated styling from mixed inline CSS to Tailwind CSS v4 in two phases: shell (App, LeftPanel, RightPanel chrome), then slide internals, dock, digest carousel, and avatar. Kept karaoke keyframes as custom CSS because they are dynamic.
- Added product UX: skeleton loaders, keyboard navigation, sidebar toggle, play/progress animation, notification banner (including Sarvam credits exhausted), and renamed “karaoke” copy to read-along for a clearer user term.
- After laptop limits, cloned the repo onto the RDP server and wrote a project briefing (file structure, models, endpoints) so development could continue there without the local chat history.

Outcome: A consistent bilingual UI and a development environment that could host heavier models.

Week 2 — 6 July to 12 July 2026

Theme: Dual-mode GPU architecture, layout quality, security, image pipeline v1

- Refactored the backend into laptop orchestrator + Kaggle GPU server (veda_gpu_server.py) reached through a Cloudflare tunnel. Providers (LLM_PROVIDER, TRANSLATE_PROVIDER, LAYOUT_PROVIDER) support local | kaggle | auto.
- Forced load_dotenv(..., override=True) so a saved .env tunnel URL always wins over stale shell variables — a frequent operational bug.
- Replaced naive PDF text extraction with pdfplumber: two-column detection (gutter $\geq 3\%$ of page width), primary-column-first reading order, image-zone word filtering, header/footer stripping.
- Improved heading detection on the frontend using font-size gap clustering instead of a single font-size threshold.
- Introduced image-primary pages (covers/ads with few words) so those pages skip unreliable topic/LLM paths and show the visual as the slide.
- Built /upscale_image with people-aware routing (YuNet faces, optional HOG). Early versions used Real-ESRGAN; that was later removed as too heavy and poorly matched to charts.
- Security pass: payload limits, PDF byte validation, CORS restricted to local dev origins, required strong KAGGLE_API_SECRET, optional VEDA_API_KEY, redacted tunnel URLs in /health. Centralized frontend API config in src/config/api.js.
- Split models: 3B GGUF on laptop, 7B GGUF in the Kaggle project-models bundle, with publish scripts.
- Started Layer 1 decorative image prompts on Kaggle (SmolVLM caption $\rightarrow$ Qwen prompt), then Layer 2 generation (SDXL-class img2img).

Outcome: The app could run “smart” on GPU hours and “good enough” on CPU, with magazine-quality layout extraction and a secure tunnel contract.

Week 3 — 13 July to 19 July 2026

Theme: Make GPU generation reliable; fix routing, concurrency, and language-switch regressions

- Sequential model pre-warm on Kaggle (NLLB → Qwen → SmolVLM → img2img) so the first user request would not hit Cloudflare’s ~100s timeout (HTTP 524). Image endpoints return 503 while pre-warm is in progress; the laptop then falls back to Lanczos instead of hanging.
- Switched default img2img from full SDXL (~6 GB) to Segmind SSD-1B (~3 GB) for faster cold start on T4 GPUs. Strength kept low (~0.35) so the PDF image remains the composition.
- Implemented Phase 1 subject routing: charts/screenshots/text-heavy → Lanczos always; people → img2img; tiny photo-like decorative → img2img; oversized or uncertain → Lanczos.
- Bug: img2img logged but never called Kaggle. `kaggle_generate_image()` was nested under the `too_large` branch. I moved it under `if use_kaggle_gen` so people and photo_tiny routes actually reach the notebook.
- Bug: YuNet crash on parallel upscales. Wrapped `detect()` in a thread lock because the frontend fires concurrent `/upscale_image` calls.
- Bug: Telugu toggle reverted upscaled images. English pageCache was saved before async upscale finished. I added `applyUpscaledImages`, patched cache on completion, and kept `upscaledUrlMapRef` across EN↔TE on the same page.
- Bug: dark mode inverted body styles. `data-theme` lived on an inner div. Synced theme to `html[data-theme]` in `PDFContext.jsx` and fixed Tailwind’s dark variant.
- Improved digest pages (multi-article spreads): LLM summaries 40–55 words, narration 80–100 words, merge LLM text with extra PDF sentences, match images by vertical zone and title Jaccard similarity.
- Added magazine Phase 1 text post-processing: merge stacked headlines, collapse duplicate title fragments, structured HEADLINE/BODY/CALLOUT tags, Q&A page layout for `/analyze`.
- Frontend cache improvements in `LeftPanel/RightPanel` so revisiting a page is faster.

Outcome: Decorative generation became an actual path (not only a log line), bilingual image state stayed consistent, and digest/magazine pages read more like articles than broken slides.

Week 4 — 20 July to 21 July 2026

Theme: Cover/background art and shareable HTML reading view

- Full-page background images had been dropped (width > 90% of page). Cover pages therefore showed zero images. I tagged backgrounds with `isBackground: true`, gated extraction with `EXTRACT_BACKGROUND_IMAGES`, and merged local backgrounds into Kaggle layout responses until the bundle was republished.
- Frontend helpers so carousels use inset photos, covers can use the background, digest strips ignore wallpaper, and backgrounds skip expensive upscale.

- Built Open HTML export: self-contained Medium/Substack-style page in a new tab, including digest layout, highlights, and in-session cached audio/read-along when available.

Outcome: Magazine covers display as intended, and a single page can be previewed as a readable article without the app chrome.

5. Challenges, Problems, and Solutions

Challenge	What went wrong	How I solved it
Telugu UI did not switch	Translation covered narration only; stale English cache	Translate all slide fields; NLLB primary; cache keys per language
Laptop cannot run 7B / SDXL	CPU-only Windows machine, heavy RAM	Dual-mode: 3B + Lanczos locally; 7B + SSD-1B on Kaggle T4 × 2
Tunnel URL changes every notebook	Stale KAGGLE_API_BASE_URL in the shell	load_dotenv(override=True); save .env then restart
Cloudflare 524 on first image gen	Cold HuggingFace downloads > ~100s	Sequential pre-warm; HF cache; 503 until ready
Pillow / transformers on Kaggle	Pillow 12 vs old stack; hub 1.x break	Pin transformers/hub; fix stack at notebook start
img2img never ran	Call nested under wrong routing branch	Single use_kaggle_gen gate; debug on laptop logs
YuNet crash under load	Parallel OpenCV detect()	Thread lock around face detect
Charts looked AI-redrawn	Blind SR / txt2img drifted	Informational → Lanczos; decorative → img2img from original pixels
Two-column magazines read wrong	PyMuPDF ignored gutters	pdfplumber column scan; primary column first
Covers had no image	Backgrounds filtered as too large	isBackground tag + merge + display helpers
Dark mode inverted	Theme attribute on inner wrapper	Sync html[data-theme] and body CSS
Upscale vanished on Telugu	Cache written before async upscale	Upscale map ref + patch all language cache keys
Sarvam 403 / empty credits	Placeholder or exhausted key	User notification; TTS fallback to edge-tts
Small LLM JSON invalid	3B output often malformed	JSON repair; structured prompts
GPU hours / 530 tunnel down	Notebook stopped	Auto fallback to local providers
OpenCV 5 missing HOG	Person detect failed	Face-only YuNet path when HOG is absent

6. Improvements Delivered

Product

- English ↔ Telugu presentation with proper fonts and field-level translation.
- Read-along narration with Sarvam timestamps, silence-adjusted; proportional fallback if STT chunks words poorly.
- Digest and image-primary layouts for magazines, not only textbook-style pages.
- Per-page HTML reading export.
- Clearer loading, navigation, and API-credit feedback.

Engineering

- Shared layout/analyze code between laptop and Kaggle.
- Provider flags and health endpoint that report mode without leaking the tunnel URL.
- Dataset publish workflow for project-models (7B GGUF + GPU server scripts).
- Conservative image router that protects charts and spends GPU only on people/tiny photos.
- Security baseline suitable for a local/RDP demo (CORS, PDF validation, secrets not in git).

Quality of output

- Column-aware extraction and stacked-headline merge for magazine titles.
- LLM prompts that distinguish BODY vs CALLOUT vs digest topics.
- Img2img that keeps original composition instead of inventing a new picture.

7. Technical Snapshot (End of Internship Build)

Frontend: React 19, Vite 8, Tailwind v4, Context + useReducer, PDF.js utilities. Backend: FastAPI tts_server.py (port 8765). Models: Qwen 2.5 Instruct (3B local / 7B Kaggle GGUF), NLLB-200, SmolVLM-500M, SSD-1B img2img, YuNet ONNX. Speech: Sarvam TTS → Sarvam STT word boundaries → silence alignment; edge-tts fallback. Ops: Kaggle notebook + cloudflared; laptop always the user-facing API.

Pipeline in production use:

- Upload PDF (mirrored to Kaggle if layout provider is remote).
- GET /page_layout → frontend post-process (headlines, digest, image-primary).
- POST /analyze → slide JSON.
- Optional Telugu via /translate_deck / field endpoints.
- Play → /tts + word boundaries.
- Images → /upscale_image (Lanczos or Kaggle img2img).
- Optional Open HTML export.

8. Work Still Out of Scope

These items were identified but not completed in this period:

- Zoro talking-photo avatar (HeyGen) and video cache.
- Disk cache for TTS, analyze, and generated images (frontend cache exists only in-session).
- Phase 2 GPU layout parser (Docling/Marker) for the hardest decorative spreads.
- Export-all-pages / download HTML and batch TTS on export.
- Raising img2img coverage (many photos still route to Lanczos because of overlapWordCount).

9. What I Learned

- Routing is as important as the model. A 7B model and SSD-1B do not help if the request never leaves the laptop or if charts are sent to img2img.

- Fallbacks must preserve user intent. A TTS fallback is fine; showing English while Telugu is selected is not.
- Async UI state needs a cache strategy. Language switches and background upscale will fight each other unless maps and cache keys are designed together.
- Cloud notebooks are not servers. Tunnel URLs, pre-warm, quota, and 524/530 errors are part of the product design, not ops trivia.
- Magazine PDFs are a layout problem first. Font-size gaps, columns, backgrounds, and callouts matter more than prompt wording for many pages.

10. Conclusion

Over four intensive weeks I moved Veda from a local prototype (PDF → LLM slide → Sarvam speech) to a resilient bilingual presentation system with magazine-aware layout, GPU offload, and honest fallbacks. Week 1 established UX and Telugu correctness under hardware limits. Week 2 introduced the Kaggle dual-mode architecture and layout/security foundations. Week 3 made image generation and caching reliable. Week 4 restored cover art and added HTML export.

The system is usable with or without GPU hours, which was the internship’s central engineering requirement.

Appendix A — Source of this report

- Local Cursor chats (workspace WEB APP): project onboarding, Telugu bugs, Tailwind phases, Sarvam notifications, RDP handoff (29 June – 1 July 2026).
- Git history on main (29 June – 21 July 2026).
- Project architecture log (.cursorrules), including dated incident write-ups for img2img routing, YuNet, dark mode, backgrounds, and Kaggle pre-warm.

Appendix B — Representative commit themes by week

Week	Dates	Representative commits
1	29 Jun – 1 Jul	Initial docs and gitignore; Tailwind UI; loading/keyboard UX; NLLB Telugu + Noto Sans Telugu
2	2–10 Jul	pdfplumber + upscaling; dual-mode Kaggle; HOG/YuNet routing; security + bundle; Layer 1/2 image gen
3	14–17 Jul	Pre-warm; SSD-1B; digest prompts; subject routing; panel caches
4	21 Jul	Background images; HTML export

End of report